提高组模拟赛题解

第 25 套

精灵（elf）

部分分

$n \leq 20$ 时枚举哪些精灵单独成组，其余精灵排序后首尾配对。$n \leq 500$ 时可以使用下面的做法，每次加入零后重新排序，时间复杂度为 $O(n^2 \log n)$。

正解

让一个精灵单独成组，等价于让它与一个能力值为 0 的虚拟精灵配对。枚举加入的零的个数 k，其中 $0 \leq k \leq n$ 且 $n + k$ 为偶数，然后将所有精灵两两配对。

对于一个已经排好序的偶数长度序列，总能首尾配对。设最小值 x、最大值 y 原来分别与 u, v 配对，将这两对改为 (x, y)、(u, v)，两个新和都位于 $x + u$ 与 $v + y$ 之间，组和的最大值不会变大，最小值不会变小。反复交换，即可得到首尾配对的方案。

如果两个虚拟精灵配成一组，删去这组即可，极差不会变大；而任何合法分组都能通过给单独成组的精灵补零转化为某次枚举。因此，所有 k 中的最小极差就是答案。

原序列只需排序一次。加入的零总在负数与非负数之间，按下标分段访问即可，不必重新排序或复制整个序列。每次枚举扫描 $O(n)$ 对，时间复杂度为 $O(n^2)$，空间复杂度为 $O(n)$。

排列（permutation）

部分分

对每个子区间计算前缀最大值与后缀最小值，找出能独立排序的各段，时间复杂度为 $O(n^3)$。也可以枚举切分位置，利用左右最值的单调性双指针统计所有合法左右端点，将时间降为 $O(n^2)$。

正解

排序一个长度为 t 的区间需要 $t-1$ 的代价。相交的操作可以合并成一次，代价不增，因此最优方案可看作将序列划分成若干段，分别排序。相邻两段之间能切开，当且仅当左段最大值小于右段最小值。

于是子区间 $[l,r]$ 的最小代价等于 $r-l$ 减去内部合法切分位置数。第一部分对所有子区间的和为

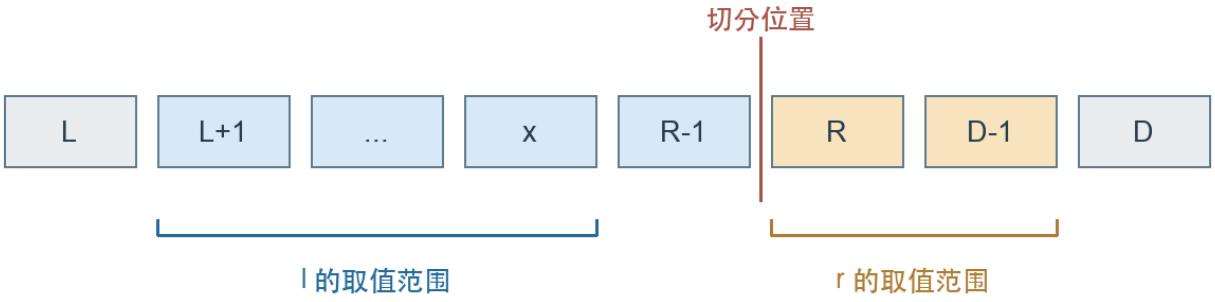
$$\sum_{l\le r}(r-l)=\frac{n(n-1)(n+1)}{6}.$$

下面统计需要减去的切分三元组 (l,c,r)，满足 $\max p[l\dots c]<\min p[c+1\dots r]$。

按左段最大值所在位置 x 分类。设 L,R 是 x 左右第一个大于 p_x 的位置，缺少时分别取 $0,n+1$。若 $R=n+1$，右段无处开始，没有贡献；否则切分位置只能是 $c=R-1$：再向左会让右段包含小于 p_x 的数，再向右会让左段包含大于 p_x 的数。

再设 D 是 R 右侧第一个小于 p_x 的位置，不存在时取 $n+1$。左端点可取 $L+1\dots x$，右端点可取 $R\dots D-1$，贡献为

$$(x-L)(D-R).$$



图中切分点固定在 $R-1$，蓝色、黄色括号分别表示左右端点的选择范围。

按数值从小到大处理位置 x，用双向链表保存尚未删除的位置，删除前的左右邻居就是 L,R；用有序集合保存已经处理的位置，查询大于 R 的最小位置就是 D。逐项扣除贡献即可。时间复杂度为 $O(n\log n)$，空间复杂度为 $O(n)$。

魔法（magic）

部分分

枚举非空前缀与非空后缀的拼接，用字符串集合或哈希判重。原有的单字母咒语另行加入。

正解

将所有原咒语正序、倒序分别插入两棵 Trie，第一棵的非根节点对应不同的非空前缀，第二棵对应不同的非空后缀。

一个新串可能有多个合法切分，只统计切分位置最靠右的一次。设当前前缀对应第一棵 Trie 的节点 u，后缀首字母为 c。若 u 没有字符 c 的儿子，就不能继续把后缀的首字母移到前缀，所有以 c 开头的后缀均可计入。若存在这个儿子，长度至少为 2 的后缀还能继续右移切分，因此只计入长度为 1、等于 c 的后缀。

例如 `abcd` 同时可以切成 `a|bcd` 与 `ab|cd`。如果 `abc` 也是合法前缀，还能继续切成 `abc|d`；只保留最后一个合法位置，便不会重复统计。前缀集合向左封闭，后缀集合向右封闭，合法切分位置连续，所以上述判断不会漏掉中间的切分。

倒序 Trie 中入边为 c 的非根节点数，就是以 c 开头的不同后缀数；是否存在长度为 1 的后缀 c，只需检查是否有原咒语以 c 结尾。遍历第一棵 Trie 的每个非根节点及 26 个字母即可求和。长度至少为 2 的原咒语已经能由自己的前缀、末字母拼出，最后只需补上不同的单字母原咒语。

设输入总长度为 S，时间、空间复杂度均为 $O(26S)$。

宴会（feast）

部分分

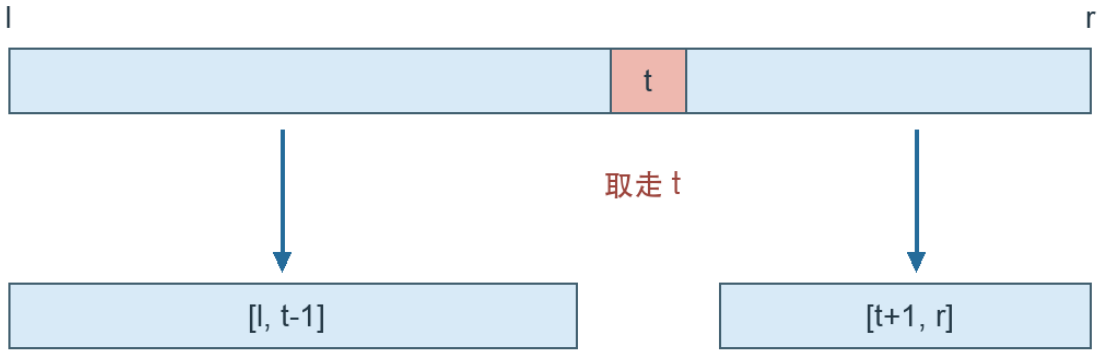
枚举所有长度在 $[L,R]$ 内的区间，排序后取最大的 k 个区间和。$k=1$ 时，维护合法左端点对应的最小前缀和即可线性求解。

正解

设前缀和为 s_i。固定左端点 o，合法右端点范围为 $[o+L-1,\min(n,o+R-1)]$，对应区间和为 s_t-s_{o-1}。用 ST 表预处理前缀和最大值的位置，就能在 $O(1)$ 时间找到任意一段候选右端点中的最优位置。

用 (o,l,r) 表示左端点固定为 o、右端点还可取 $l\dots r$ 的一批区间，把这一批的最大区间和放入大根堆。最初对每个左端点加入完整的合法范围，堆顶便是当前所有候选区间中的最大值。

弹出 (o,l,r)，设它的最优右端点为 t，将对应区间和加入答案。这个右端点不能再选，其余候选恰好分为 $[l,t-1]$ 和 $[t+1,r]$，分别非空时重新入堆。每次拆分既不丢失候选，也不重复选中同一区间，连续弹出 k 次就是所求。



两侧非空的范围分别重新入堆

图中固定左端点不变，选走右端点 t 后，只把两侧尚未选过的右端点范围放回堆中。

预处理时间为 $O(n\log n)$，堆中元素数量为 $O(n+k)$，总时间复杂度为 $O(n\log n+(n+k)\log(n+k))$，空间复杂度为 $O(n\log n+k)$。